



TicketMet

Micro-projet Web

Jessy Fallavier

21500607

Table des matières

1	Déploiement	1
2	Sujet de l'application	1
3	Fonctionnalités	1
4	Cas d'utilisation	1
5	Schéma de base de données (server/main/schema.sql)	2
6	API Web	3
6.1	Ticketmaster	3
6.2	Setlist.fm	3
7	Mise à jour	4
7.1	Sync Ticketmaster	4
7.2	Radar d'alertes	4
7.3	Setlist	4
8	Requêtes implémentées	5
8.1	Routes de consultation	5
8.2	Routes d'authentification	6
8.3	Routes d'interaction et profil	6
9	Description serveur	7
9.1	Architecture du serveur	7
9.2	Tableau des jobs	8
9.3	Mail applicatif	9
10	Description client	10
10.1	Architecture du client	10
10.2	Plan du site par écran	11
10.2.1	searchView	11
10.2.2	detailView	12
10.2.3	accountView	13
11	Schéma global du système	14
12	Perspectives futures	15
13	Références et IA	15

1 Déploiement

- Le projet est conteneurisé avec Docker. C'est déployé sur mon VPS déjà en place, sans frais supplémentaires pour l'UE.
- Code source : github.com/jessyfal04/ticketmet.
- Site : ticketmet.jessyfal04.dev.
- Les clés API sont lues depuis `.secrets/ticketmaster.mk`, chargé par le `Makefile`.

2 Sujet de l'application

TicketMet est une application web qui agrège les concerts en Europe et centralise leurs données importantes, comme la date, la salle, le plan et la setlist potentielle. L'idée est de garder un point d'entrée unique pour suivre les concerts, retrouver les informations utiles rapidement et éviter de jongler entre plusieurs sites.

L'application ajoute aussi une couche sociale. On peut suivre ses favoris avec une alerte de vente, recevoir des alertes de nouveauté, et utiliser les SNS (Social Network Service) pour faire du WTB (Want To Buy) / WTS (Want To Sell) ou simplement se retrouver avant un concert.

3 Fonctionnalités

- Suivre des concerts, des artistes et des salles
- Consulter la liste des concerts en Europe
- Rechercher un concert par artiste ou salle
- Ouvrir la fiche d'un concert avec la date, la salle, la localisation, le plan si dispo et la setlist potentielle
- Utiliser Ticketmaster pour peupler la base de concerts
- Utiliser Setlist.fm pour enrichir une fiche quand la setlist est disponible
- Ajouter un concert en favoris pour activer l'alerte de vente
- Se mettre en WTB ou en WTS sur un concert
- Voir les autres personnes en WTB ou WTS pour un concert
- Activer une alerte de nouveaux concerts pour un artiste ou une salle
- Recevoir un mail de bienvenue à l'inscription
- Recevoir automatiquement des mails d'alertes
- Se connecter avec une passkey
- Renseigner ses SNS sur son profil
- Voir les SNS des gens qui vont au même concert

4 Cas d'utilisation

- Scénario 1 : Alice met un concert en WTS, Bob se met en WTB ; ils reçoivent un mail de match, consultent la fiche du concert, récupèrent les SNS et se contactent pour l'échange de place.
- Scénario 2 : Eve active une alerte nouveauté sur NMIXX, reçoit un mail d'un nouveau concert, le met en favoris, puis reçoit l'alerte de vente quand les ventes ouvrent.

- Scénario 3 : Dave cherche un concert par salle, ouvre la fiche, consulte la setlist potentielle et ajoute le concert en favoris pour être alerté au lancement des ventes. Le jour des ventes il échoue à acheter une place, il se met en WTB.
- Scénario 4 : Lina consulte un concert depuis la recherche, elle copie le lien direct du concert puis l'envoie à son amie Lisa. Lisa ouvre la fiche avec le lien profond, consulte le plan de la salle et achète sa place.
- Scénario 5 : Sam se connecte avec une passkey, met à jour ses SNS dans son profil et retrouve ensuite les personnes qui vont au même concert.

5 Schéma de base de données (server/main/schema.sql)

Le schéma sépare les concerts, les utilisateurs, les relations sociales, les alertes et l'état de synchronisation comme tel.

- **artists** : #id INTEGER / name TEXT NN
- **venues** : #id INTEGER / name TEXT NN / city TEXT NN / country TEXT NN
- **concerts** : #id INTEGER / name TEXT NN / date TEXT NN / #venue_id INTEGER NN ref(venues(id)) / #artist_id INTEGER NN ref(artists(id)) / url TEXT NN / photo_url TEXT / seatmap_url TEXT / sale_start_datetime TEXT NN / created_at INTEGER
- **users** : #id INTEGER / email TEXT NN UNIQUE / password_hash TEXT NN
- **sessions** : #id INTEGER / #user_id INTEGER NN ref(users(id)) / token_hash TEXT NN UNIQUE / expires_at INTEGER NN
- **webauthn_credentials** : #id INTEGER / #user_id INTEGER NN ref(users(id)) / credential_id TEXT NN UNIQUE / public_key TEXT NN / sign_count INTEGER / user_present BOOLEAN / user_verified BOOLEAN / backup_eligible BOOLEAN / backup_state BOOLEAN
- **webauthn_challenges** : #id INTEGER / user_id INTEGER ref(users(id)) / token_hash TEXT NN UNIQUE / kind TEXT NN CHECK (kind IN ('registration', 'login')) / session_data TEXT NN / expires_at INTEGER NN
- **user_sns** : #user_id INTEGER NN ref(users(id)) / #sns TEXT NN
- **wt** : #user_id INTEGER NN ref(users(id)) / #concert_id INTEGER NN ref(concerts(id)) / type TEXT NN CHECK (type IN ('wtb', 'wts'))
- **favorites** : #user_id INTEGER NN ref(users(id)) / #concert_id INTEGER NN ref(concerts(id)) / created_at INTEGER
- **alerts** : #id INTEGER / #user_id INTEGER NN ref(users(id)) / target_type TEXT NN CHECK (target_type IN ('artist', 'venue')) / target_id INTEGER NN / created_at INTEGER / UNIQUE(user_id, target_type, target_id)
- **notifications** : #dedupe_key TEXT
- **setlists** : #concert_id INTEGER NN ref(concerts(id)) / #song_order INTEGER NN / song_name TEXT NN
- **sync_ticketmaster** : #id INTEGER CHECK (id = 1) / max_visibility TEXT NN

6 API Web

J'utilise deux API externes, chacune avec un rôle précis : Ticketmaster pour peupler la base de concerts, puis Setlist.fm pour enrichir la fiche d'un concert quand la setlist est disponible.

6.1 Ticketmaster

- Source des concerts, des images, des dates, des salles et des artistes.
- TicketMet interroge `events.json` par pays, filtre les nouveautés avec `publicVisibilityStartDateTime`, puis conserve les champs utiles pour construire la fiche concert.
 - Champs utiles :
 - Identifiant de l'événement : `id`
 - Nom de l'événement : `name`
 - Lien public : `url`
 - Images : `images[].url`
 - Date de concert : `dates.start.localDate / localTime / dateTime`
 - Ouverture des ventes : `sales.public.startDateTime`
 - Plan de salle : `seatmap.staticUrl`
 - Salle : `_embedded.venues[0].id / name / city.name / country.countryCode`
 - Artiste : `_embedded.attractions[0].id / name`

6.2 Setlist.fm

- Source d'une setlist potentielle par artiste.
- TicketMet interroge `search/setlists`, prend la première setlist qui contient des songs, puis reconstruit la liste des titres à partir des chansons.
 - Champs utiles :
 - Liste de setlists `setlist[]`
 - Noms des chansons d'une setlist `sets.set[].song[].name`

7 Mise à jour

Le rafraîchissement des données est découpé en jobs arrière-plan. Le serveur HTTP ne bloque pas sur ces traitements ; il sert les routes, pendant que les workers maintiennent la base et les notifications à jour.

7.1 Sync Ticketmaster

- Le marqueur `publicVisibilityStartDateTime` sert de filtre de nouveauté, stocké dans `SyncTicketmaster.max_visibility`.
- Les concerts sont récupérés puis insérés par pays, avec les pays frontaliers à la France : DE, FR, BE, NL, LU, CH, AD, GB, IT, MC et ES. Les données françaises sont limitées car **Ticketmaster France** est en réalité **Ticketnet**, dont le site est différent.
- Le job tourne en goroutine, démarre tout de suite, puis se relance toutes les 15 minutes avec la clé Ticketmaster fournie par le shell `TICKETMASTER_API_KEY`.
- Un nettoyage est effectué : on garde les events avec un nom d'événement, une salle ou un artiste exploitable, une seule photo avec bonus 16 :9 si possible, et on ignore les dates de vente aberrantes avant 2000.

7.2 Radar d'alertes

- Le radar vérifie les alertes de nouveauté, les alertes de vente et les matches WTB/WTS.
- Les notifications déjà envoyées sont dédoublées via `notifications.dedupe_key`.
- Quand un événement déclenche une alerte, le worker prépare le mail correspondant sans bloquer les handlers HTTP.

7.3 Setlist

- La setlist est d'abord lue dans le cache SQLite.
- Si elle manque et que `SETLISTFM_API_KEY` est fourni, le serveur interroge Setlist.fm puis complète la réponse.

8 Requêtes implémentées

8.1 Routes de consultation

TABLE 1 – Routes de consultation

REQUÊTES	DESCRIPTION	RÉPONSE
GET /healthz	healthcheck serveur	texte brut ok, code 200
GET /api/concerts avec artistID, venueID, country, status et page	liste des concerts, avec filtre optionnel par artiste, salle, pays et pagination	liste de DisplayConcert
GET /api/concerts par identifiant	détail d'un concert	DisplayConcert
GET /api/artists	liste des artistes	liste d'Artist
GET /api/venues	liste des salles	liste de Venue
GET /api/favorites par concert	voir les SNS des gens qui vont au même concert	SNS des personnes du même concert
GET /api/setlist par concert	voir la setlist potentielle d'un concert	liste de chansons potentielles
GET /api/wt par concert	voir les WTB et WTS liés à un concert	WTB, WTS et compteurs

8.2 Routes d'authentification

TABLE 2 – Routes d'authentification

REQUÊTES	DESCRIPTION	RÉPONSE
POST /api/auth/register	créer un compte email/password et ouvrir une session cookie	utilisateur connecté
POST /api/auth/login	se connecter avec email/password	utilisateur connecté
POST /api/auth/logout	révoquer la session courante	statut 204
DELETE /api/auth/unregister	supprimer son compte avec confirmation password	statut 204
GET /api/auth/me	récupérer l'utilisateur connecté	utilisateur connecté
GET /api/auth/email-exists avec email	vérifier si un email existe	oui ou non
POST /api/auth/passkeys/register/begin	commencer l'ajout d'une passkey pour l'utilisateur connecté	options WebAuthn
POST /api/auth/passkeys/register/finish	valider et stocker la passkey créée	confirmation OK
POST /api/auth/passkeys/login/begin	commencer une connexion passkey sans email	options WebAuthn
POST /api/auth/passkeys/login/finish	valider la passkey et ouvrir une session cookie	utilisateur connecté
GET /api/auth/passkeys	lister les passkeys de l'utilisateur connecté	liste de passkeys
DELETE /api/auth/passkeys par credentialId	supprimer une passkey de l'utilisateur connecté	statut 204

8.3 Routes d'interaction et profil

TABLE 3 – Routes d'interaction et profil

REQUÊTES	DESCRIPTION	RÉPONSE
POST ou DELETE /api/favorites par concert	ajouter ou retirer un favori ; cela active ou désactive l'alerte de vente	statut 204
POST ou DELETE /api/wt par concert avec type wt ou wts	se mettre ou retirer en WTB ou WTS ; POST remplace l'autre statut du même user sur le même concert, et un concert expiré renvoie 409	statut 204
POST /api/alerts avec targetType et targetId	créer une alerte de nouveauté	confirmation OK
DELETE /api/alerts par alertId	retirer une alerte	statut 204
GET /api/me	voir le profil complet	profil complet
PATCH /api/me	modifier le profil, ici les SNS	profil mis à jour

9 Description serveur

Le serveur Go s'appuie sur `net/http` pour l'HTTP et sur `database/sql` pour SQLite. J'ai gardé les routes proches des données, et les requêtes SQL restent visibles dans les handlers et dans les jobs.

Le démarrage se fait dans `server/main/main.go`. Ce fichier ouvre la base, prépare les canaux et lance les workers en arrière-plan avant de servir HTTP. La façade des routes reste dans `server/api/api.go`, qui assemble les handlers au même endroit.

La base SQLite persiste dans `/app/data/ticketmet.sqlite3` en production.

Les scanners partagés vivent dans `server/model/scan.go` avec les structs dans `server/model/models.go`; ça permet garder les requêtes SQL simples et proches des données.

9.1 Architecture du serveur

```
server
|-- api
|   |-- alerts.go
|   |-- api.go
|   |-- artists.go
|   |-- auth.go
|   |-- concerts.go
|   |-- favorites.go
|   |-- healthz.go
|   |-- passkeys.go
|   |-- profile.go
|   |-- setlist.go
|   |-- venues.go
|   |-- wt.go
|-- job
|   |-- alertradar.go
|   |-- database.go
|   |-- mailserver.go
|   |-- setlistfm.go
|   |-- ticketmaster.go
|   |-- worker.go
|-- main
|   |-- main.go
|   |-- schema.sql
|-- model
    |-- models.go
    |-- scan.go
```

9.2 Tableau des jobs

TABLE 4 – Tableau des jobs

Fichier	Déclenchement	Rôle	Flux
server/job/database.go	en continu	exécute les requêtes SQL en série	dbChan
server/job/ticketmaster.go	au démarrage puis toutes les 15 min	synchronise les concerts Ticketmaster	HTTP GET + dbChan
server/job/setlistfm.go	à la demande via /api/setlist/id	lit le cache puis interroge Setlist.fm si besoin	setlistChan + dbChan + HTTP GET
server/job/alertradar.go	au démarrage puis toutes les 1 min	détecte les alertes, les ventes et les matches WTB/WTS	dbChan + mailChan
server/job/mailserver.go	en continu	consomme les mails en file buffered et les envoie en SMTP	mailChan + SMTP

9.3 Mail applicatif

Le mailservice envoie le mail de bienvenue à l'inscription, puis les mails d'alertes du radar.

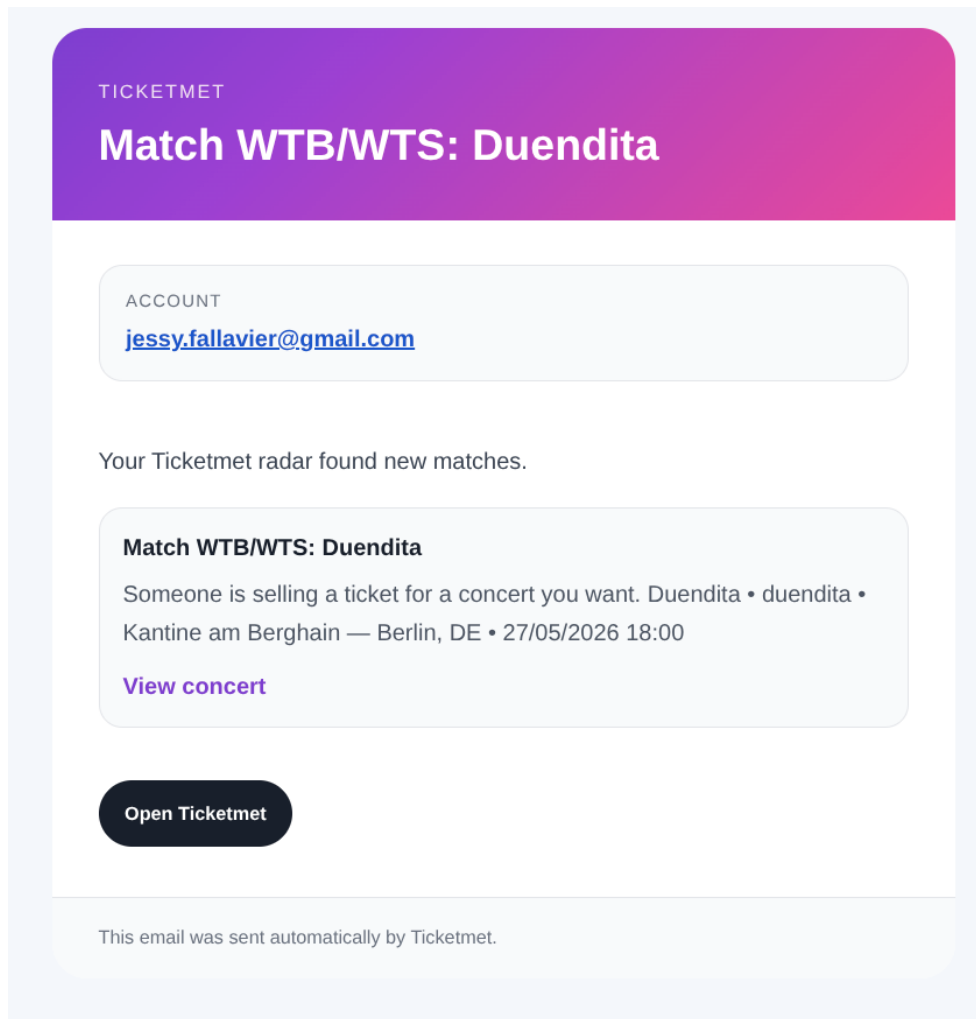


FIGURE 1 – Mail de bienvenue et mail d'alertes

10 Description client

Le client est une monopage en JavaScript : jQuery gère les appels AJAX, les événements DOM et le scroll ; Bulma CSS porte la structure visuelle (les forms, les buttons, les tags et les blocs de notifications) ; React / ReactDOM restent limités à quelques islands ciblées. Le code est découpé entre `client/lib/script.js` pour l'état partagé et les helpers, puis `client/lib/account.js`, `client/lib/search.js`, `client/lib/detail.js`, `client/lib/passkeys.js` pour les écrans spécifiques.

10.1 Architecture du client

```
client
|-- img
|   |-- concert.png
|-- index.html
|-- lib
|   |-- account.js
|   |-- detail.js
|   |-- passkeys.js
|   |-- script.js
|   |-- search.js
|-- style
    |-- bulma.min.css
    |-- style.css
```

10.2 Plan du site par écran

10.2.1 searchView

Le point d'entrée de l'application affiche les concerts, les filtres et la pagination. Au chargement, l'app vérifie l'état du serveur, la session et la disponibilité des listes de base, puis elle lance la première recherche.

- Les filtres artiste, salle, pays et période relancent la recherche.
- Le scroll déclenche le chargement de la page suivante.
- Le bouton **Clear** remet l'état à zéro.
- Le bouton **Open** d'une carte ouvre la fiche concert.
- AJAX : GET /healthz, GET /api/auth/me, GET /api/artists, GET /api/venues, GET /api/concerts?artistID=...&venueID=...&country=all|<code>&status=future|all&page=...
- AJAX : POST /api/alerts?targetType=artist|venue&targetId=... pour créer une alerte depuis la recherche

Concert search
Filter by artist, venue, country, and time window. Clear

Artist **Venue** **Country** **Time**

aespa (#507848611) All venues Great Britain Future

Alerts
Create an alert from the current search filters.

Artist: aespa
Venue: All venues

Remove artist alert Alert this venue

aespa LIVE TOUR - SYNK : COMPLæXITY - Venue Premium Tickets
aespa · AO Arena
01/14/2027, 07:00 PM
Upcoming Sale open
Open

aespa LIVE TOUR - SYNK : COMPLæXITY - in MANCHESTER
aespa · AO Arena
01/14/2027, 07:30 PM
Upcoming Sale open
Open

aespa LIVE TOUR - SYNK : COMPLæXITY - in LONDON
aespa · The O2
01/16/2027, 07:30 PM
Upcoming Sale open
Open

FIGURE 2 – Vue searchView

10.2.2 detailView

La fiche concert s'ouvre depuis la recherche ou via le lien profond `?concert=...`. Elle rassemble le détail du concert, la setlist potentielle, les SNS, les favoris et les actions WTB/WTS.

- `renderConcert()` remplit la fiche principale.
- `loadConcertFeatures()` lance les compléments autour de la setlist, des SNS et des statuts WTB/WTS.
- Les boutons **Share**, favoris, WTB et WTS déclenchent les callbacks associés.
- **AJAX** : GET `/api/concerts/{id}`, GET `/api/setlist/{id}`, GET `/api/favorites/{id}`, GET `/api/wt/{id}`
- **AJAX** : POST/DELETE `/api/favorites/{id}`, POST/DELETE `/api/wt/{id}?type=wtb|wts`
- **AJAX** : GET `/api/me` après une mise à jour de profil ou d'action si l'utilisateur est connecté

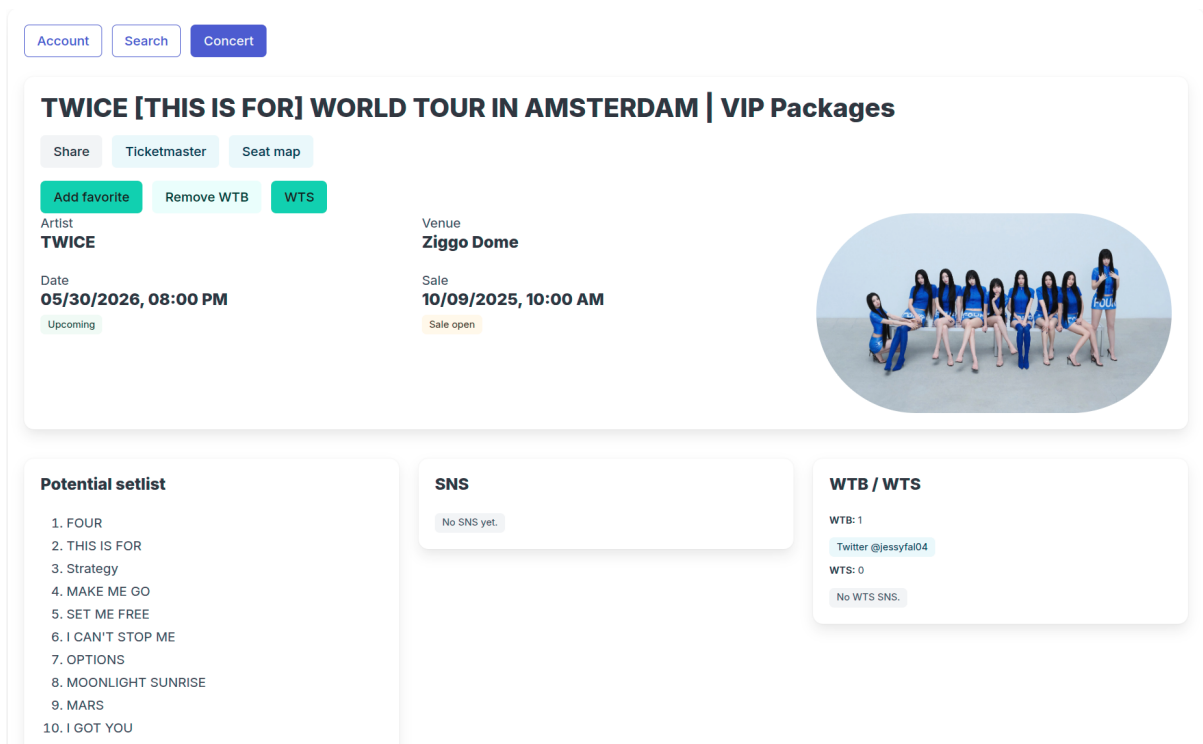


FIGURE 3 – Vue detailView

10.2.3 accountView

L'écran compte affiche soit le bloc sign in / register / passkey login, soit le profil utilisateur avec ses passkeys, ses favoris, ses WT et ses alertes.

- Quand l'utilisateur est déconnecté, il voit les formulaires d'inscription, de connexion et de connexion passkey.
- Quand l'utilisateur est connecté, il voit son profil, ses passkeys, ses favoris, ses WT, ses alertes et la zone de suppression du compte.
- `showView("accountView")` recharge les données utiles avant d'afficher la vue.
- AJAX : POST /api/auth/register, POST /api/auth/login, POST /api/auth/logout, DELETE /api/auth/unregister
- AJAX : GET /api/auth/email-exists, GET /api/auth/me, GET /api/me, PATCH /api/me
- AJAX : GET /api/auth/passkeys, POST /api/auth/passkeys/register/begin, POST /api/auth/passkeys/register/finish, POST /api/auth/passkeys/login/begin, POST /api/auth/passkeys/login/finish, DELETE /api/auth/passkeys/{credentialId}
- AJAX : DELETE /api/alerts/{alertId}

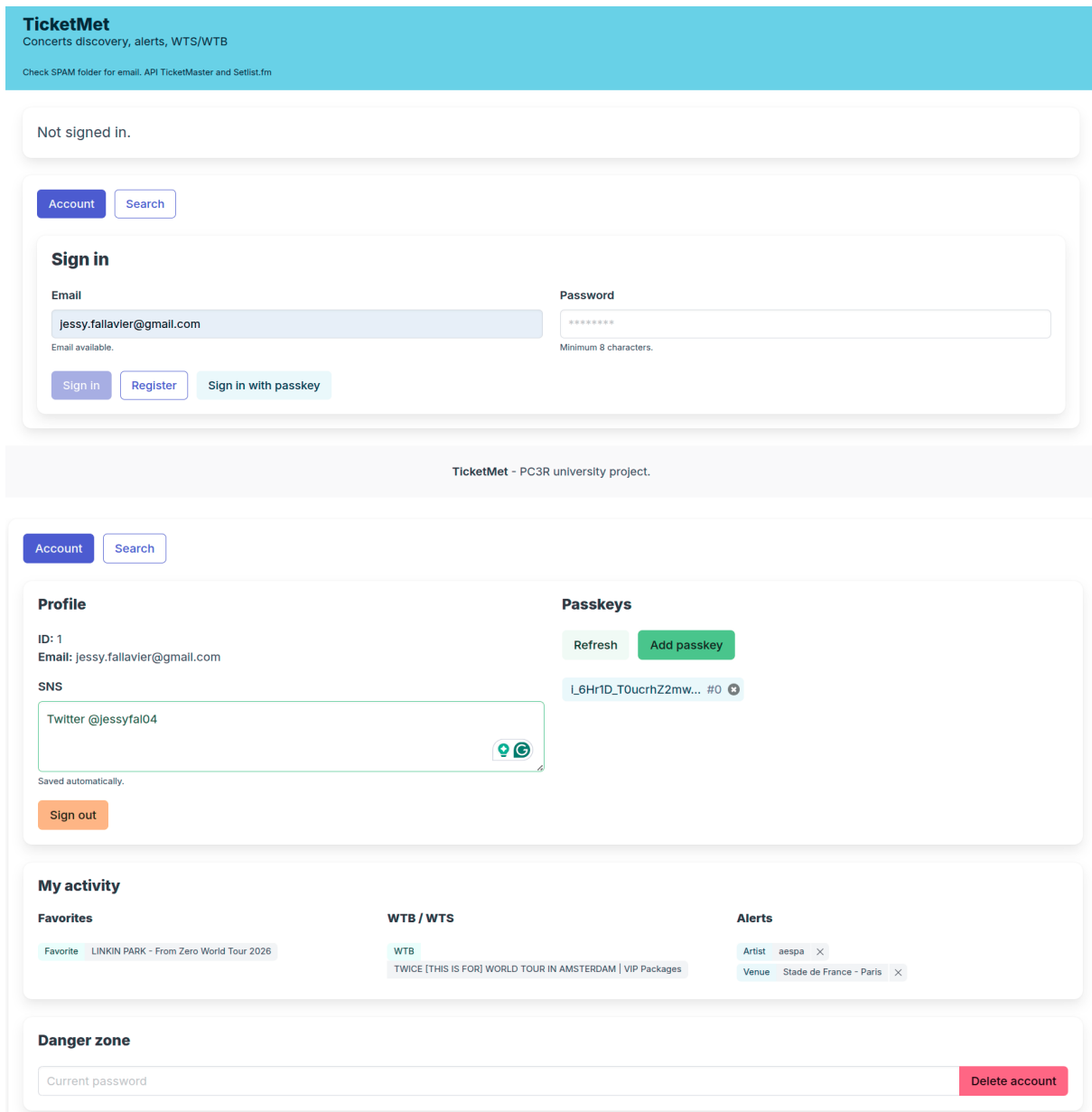


FIGURE 4 – Vue accountView non connecté (en haut) et connecté (en bas)

11 Schéma global du système

Le schéma ci-dessous résume la chaîne principale : client, AJAX, serveur, jobs, bases et APIs externes.

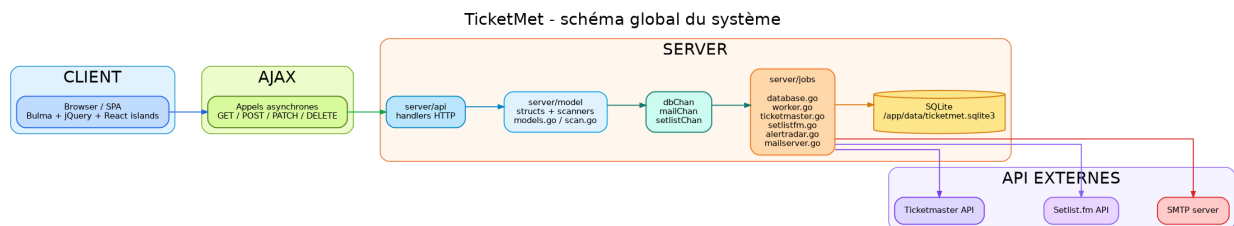


FIGURE 5 – Schéma global du système

12 Perspectives futures

- Notification push via Firebase
- Job additionnel pour retirer les sessions et les challenges passkey expirés
- Passage plus large en React ou en TypeScript, surtout sur la vue de détail d'un concert qui reste le plus gros bloc
- Mise en place d'une Progressive Web App

13 Références et IA

Mes dépôts GitHub d'anciens projets [avotey](#) pour la partie visuelle et [webauthnnotes](#) pour la partie Passkey.

Utilisation de l'IA

Dans une démarche de transparence, voici les prompts que j'ai demandé à ChatGPT.

- Explique moi comment je peux faire ce serveur en Go, la nomenclature de l'appli, comme on v
- Selon notes.md, donne des struct go qui représente ces données / id en int /
- Give a dummyData function for the app with several venues...
- Donne 2 template pour les requetes artists et venues; Donne 2 template de test.sh pour arti
- Donne la nomclature d'une fonction qui prends un string optionnel et renvoi nil ou le int
- A quoi ressemble un sync pour l'API ticketmaster dans app.md ?
- Lis le frontend que j'ai fais pour une appli de vote (docs/avotey) et applique le pour l'ap
- Clone <https://github.com/jgthms/bulma/tree/741da22368cd548111224983a1c3ece8954fe8cc/docs> de
- Quel spécification pour appli web ? genre je veux que tt sois à jour et bien faire genre bu
- Quels étapes pour déployer sur mon VPS avec Docker ? En mode auto dès que ya une modif.
- Comment ajouter du react ici, quels cas d'usage?
- Une base de données sqlite3 c bon ici nan?
- Modifie model pour que ça prenne de la db, et donne moi les requêtes sql type
- dans artist et concert et venues ya du code qui se répète nan, dans la structure. Comment f
- Il ya un mélange de nom anglais et de nom français, je ne veux garder que l'anglais.
- Automatise le resync Ticketmaster tt les 15 minutes, ça doit utiliser le schema sql et le r
- Using my 2 old projects docs/src avotey (php) and webauthnnotes (python), adapt into go, yo
- Same for the frontend I want to keep the same logic as docs/src avotey (php) and webauthnno
- Passkey inconsistency I need to store flags?
- Read the readme file, add placeholder for the remaining functions
- I prefer to have response struct instead of map, modify.
- How can I have a job with a timer via a chan?
- Imagine a beautiful html email template with a dedicated place for the mail
- Crée httpGetIntParam sous le même format que httpGetStringParam
- Do ssh vps, I have a pb the docker-deploy, it keeps restarting.
- I'll have a job folder with all the workers. You think I can mutualize some code?
- How could I create a frontend monospace Bulma inspired by avotey+webauthnnotes, I want all m
- Which scanner can be created / deleted. Maybe bool and string ?
- I really need to create a date field for the favorites alerts in order to avoid sending one
- Regarde chaque requête SQL et vérif si elle peut etre simplifiée en gardant le même comportem
- Voici mes codes javascript. Donne moi une idée de mini bouts qu'on peut mettre en mode Reac
- Here's my files, please comment each function for exemple